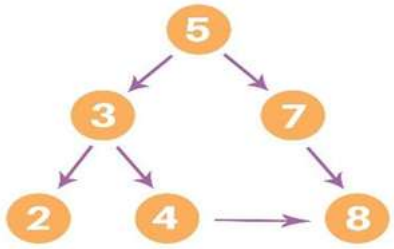


Figure: Graph for DFS implementation

```
# Activity 1: Depth First Search using recursion

graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8']
```

Run this cell to mount your Google Drive.

Learn more

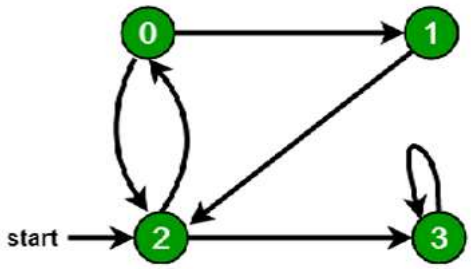
Dismiss

```
def dfs(visited, graph, node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

print("Following is the Depth-First Search:")
dfs(visited, graph, '5')
```

```
Following is the Depth-First Search:
5 3 2 4 8 7
```

```
from google.colab import drive
drive.mount('/content/drive')
```




```
# Activity 2: Depth First Traversal for a graph

from collections import defaultdict

# This class represents a directed graph using adjacency list
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)
```

```
        # Function to add an edge to graph
        def addEdge(self, u, v):
            self.graph[u].append(v)

        # A function used by DFS
        def DFSUtil(self, v, visited):
            visited[v] = True
            print(v, end=" ")

            # Recur for all adjacent vertices
            for i in self.graph[v]:
                if not visited[i]:
                    self.DFSUtil(i, visited)

        # The function to do DFS traversal
        def DFS(self):
            V = max(self.graph) + 1   # Total number of vertices
            visited = [False] * V

            # Call DFSUtil for all vertices
            for i in range(V):
                if not visited[i]:
                    self.DFSUtil(i, visited)

    # ---------------- Driver Code ----------------
    g = Graph()

    g.addEdge(0, 1)
    g.addEdge(0, 2)
    g.addEdge(1, 2)
    g.addEdge(2, 3)

    print("Following is Depth First Traversal:")
    g.DFS()
```

Run this cell to mount your Google Drive.

Learn more

```
Following is Depth First Traversal:
0 1 2 3
```

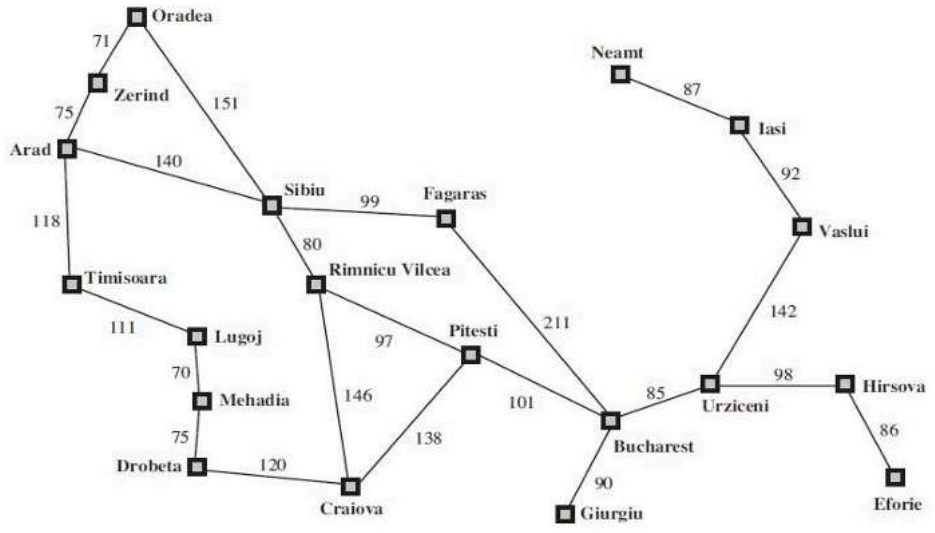


Figure: Graph for Travelling Salesman Problem

Figure: Travelling Salesman Problem

```
    # DFS for TSP (Romania Map - simplified)

    graph = {
        'Arad': [('Zerind', 75), ('Sibiu', 140), ('Timisoara', 118)],
        'Zerind': [('Arad', 75), ('Oradea', 71)],
        'Oradea': [('Zerind', 71), ('Sibiu', 151)],
        'Sibiu': [('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu Vilcea', 80)],
```

```
    'Fagaras': [('Sibiu', 99), ('Bucharest', 211)],
    'Rimnicu Vilcea': [('Sibiu', 80), ('Pitesti', 97)],
    'Pitesti': [('Rimnicu Vilcea', 97), ('Bucharest', 101)],
    'Timisoara': [('Arad', 118)],
    'Bucharest': []
}

def dfs_tsp(graph, current, goal, visited, path, cost):
    visited.add(current)
    path.append(current)

    if current == goal:
        print("Path:", " -> ".join(path))
        print("Total Cost:", cost)
        return True

    for neighbour, distance in graph[current]:
        if neighbour not in visited:
            if dfs_tsp(graph, neighbour, goal, visited, path, cost + distance):
                return True

    path.pop()
    visited.remove(current)
    return False

visited = set()
dfs_tsp(graph, 'Arad', 'Bucharest', visited, [], 0)
```

```
Path: Arad -> Zerind -> Oradea -> Sibiu -> Fagaras -> Bucharest
Total Cost: 450
```

Run this cell to mount your Google Drive.

Learn more